



**Mondragon
Unibertsitatea**

**Escuela Politécnica
Superior**

Reinforcement Learning -RL-

Markov Decision Process

The future is independent of the past given the present

Aitor Duo Zubiaurre

aduo@mondragon.edu

Olagorta Kalea, 26
48014 Bilbo (Bizkaia)

T. +34 670 254 555

www.mondragon.edu

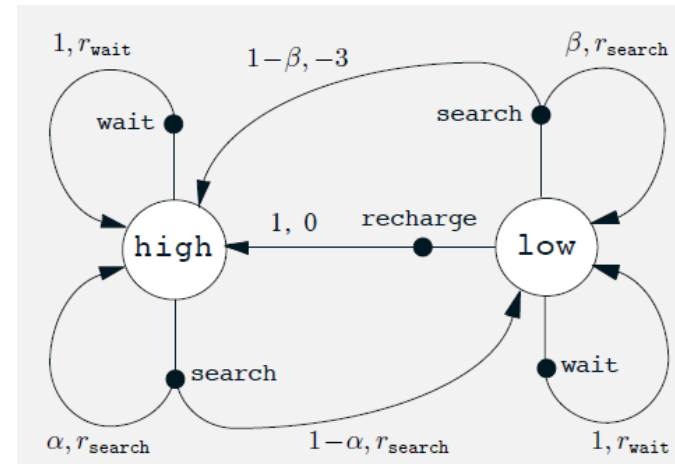
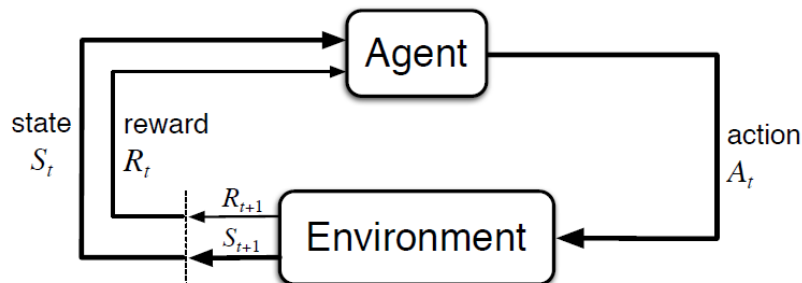
This presentation is based on:

Richard S. Sutton and Andrew G. Barto. Reinforcement learning: An introduction.
MIT Press, Cambridge, MA, 2018.

Markov Decision Process

- **Finite MDPs**

- Markov decision processes formally describe an environment for reinforcement learning
- Markov decision process is a tuple $(S, A, \{P_{sa}\}, \gamma, R)$
- S is a set of states
- A is a set of actions



Recycling robot model

Markov Decision Process

- A state S_t is Markov if and only if

$$\mathbb{P}[S_{t+1} | S_t] = \mathbb{P}[S_{t+1} | S_1, \dots, S_t]$$

- Thus, the next move depends only on where you currently are, not on how you got there.
- For a Markov state s and successor state s' , the state transition probability is defined by

$$\mathcal{P}_{ss'} = \mathbb{P} [S_{t+1} = s' | S_t = s]$$

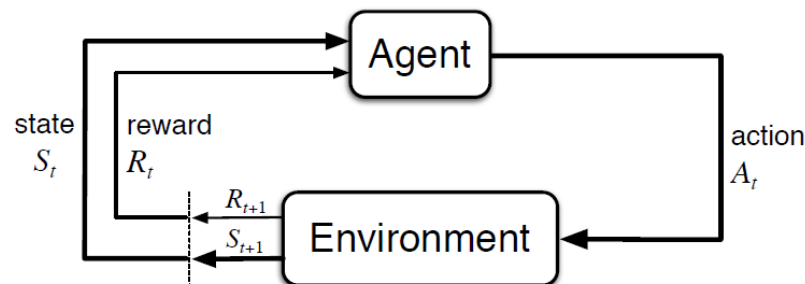
For particular values of this variable, there is a probability of those values occurring at time t , given particular values of the preceding state and action

Markov Decision Process

- **Finite MDPs**

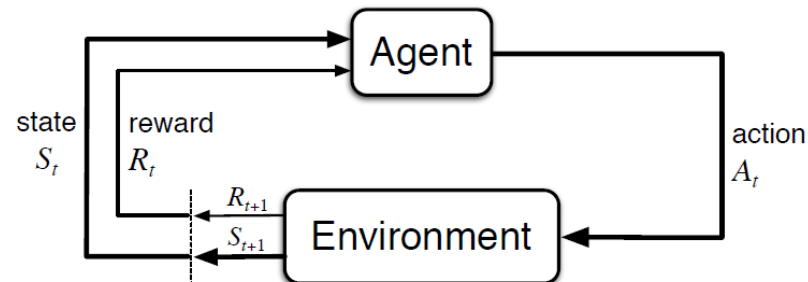
1. The agent and environment interact at each discrete time step, designated as $t = 0, 1, 2, 3, \dots$ respectively.
2. At each time step t , the agent receives some representation of the environments state, $S_t \in \mathcal{S}$,
3. Based on S_t the agent selects an action, $A_t \in \mathcal{A}(s)$.
4. In $t + 1$, because of this action, the agent receives a numerical reward, $R_{t+1} \in R \subset \mathbb{R}$, and finds itself in a new state, S_{t+1} .

$S_0, A_0, R_1, S_1, A_1, R_2, S_2, A_2, R_3, \dots$



Markov Decision Process

- **Finite MDPs**
 - In finite MDPs, states, actions and rewards have a finite number of elements.



Markov Decision Process

- **Policy**
 - At step $t = \pi_t =$
 - A mapping from states to action probabilities
 - $\pi_t(a|s)$ = probability $A_t = a$ when $S_t = s$
 - An special case when deterministic policies:
 - $\pi_t(s)$ = the action taken with prob=1 when $S_t = s$
 - From experience reinforcement learning methods specify how the agent changes its policy.
 - The agents **goal is to maximize the total reward** in a long turn.

Markov Decision Process

- **Return**

- If we have

$$R_{t+1} + R_{t+2} + R_{t+3} + \dots$$

- And the agents goal is to **maximize the total reward** in a long turn, then we want to maximize the expected return. We can have 3 cases:
 - Total reward, $G_t =$ sum of all future reward in the episode
 - Discounted reward, $G_t =$ sum of all future discounted reward
 - Average reward, $G_t =$ Average reward per time step

Markov Decision Process

- **Return**
 - **Episodic Task:** interaction breaks naturally into episodes (e.g.: plays of a game)
 - Usually used the total reward:

$$G_t = R_{t+1} + R_{t+2} + R_{t+3} + \dots + R_T$$

Markov Decision Process

- **Return**
 - **Continuing Tasks:** interaction does not have natural episodes.
 - In this case, we always use a discounted return:

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}$$

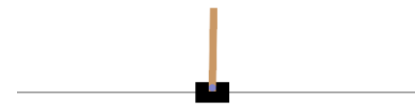
Where γ , $0 \leq \gamma \leq 1$, is the discount rate

Typically, $\gamma = 0.9$

Markov Decision Process

- **Example**

- The pendulum is placed upright on the cart and the goal is to balance the pole by applying forces in the left and right direction on the cart.
 1. Termination: Pole Angle is greater than $\pm 12^\circ$
 2. Termination: Cart Position is greater than ± 2.4 (center of the cart reaches the edge of the display)
- Episodic task (the episode ends in some termination)
 - Reward $\{+1\}$ for each step before failure
 - Return: Number of steps before failure
- Continuing task
 - Reward $\{-1\}$ in case of termination; 0 otherwise
 - Return $\{-\gamma^k\}$, for k steps before failure



Source: Gymnasium documentation

Video: <https://www.youtube.com/watch?v=Y6FVBbqjR40>

Markov Decision Process

- **Return**
 - We say that the objective is to maximize long-term reward
 - But what exactly should be maximized?

γ	Reward sequence	Return
0.5	1 0 0 0 ...	1
0.5	0 0 2 0 0 0 ...	0.5
0.9	0 0 2 0 0 0 ...	1.62
0.5	-1 2 6 3 2 0 0 0 ...	2

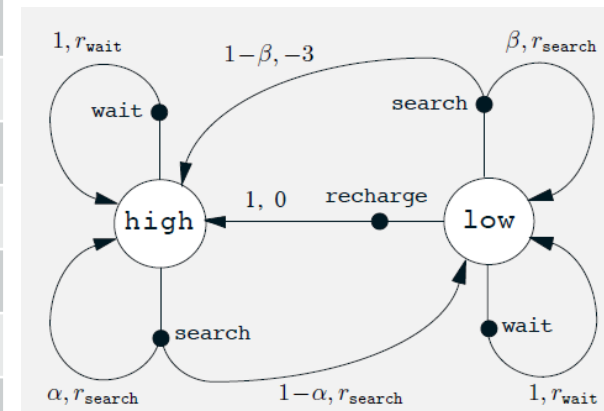
Markov Decision Process

- Suppose $\gamma = 0.5$ and the reward sequence is
 - $R_1 = 1, R_2 = 6, R_3 = -12, R_4 = 16$ and $R_t = 0$ for $t \geq 5$
 - What are the following returns?
 - $G_4 =$
 - $G_3 =$
 - $G_2 =$
 - $G_1 =$
 - $G_0 =$

Markov decision process

- Example
 - Recycling robot

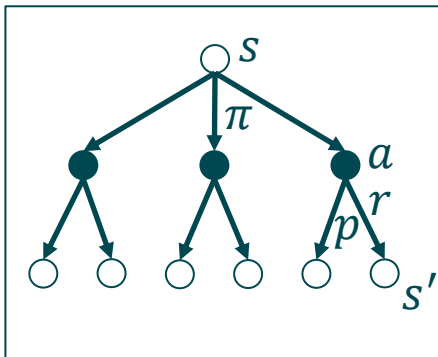
s	a	s'	$p(s' s, a)$	$r(s, a, s')$
High	search	High	α	r_{search}
High	search	Low	$1 - \alpha$	r_{search}
low	search	High	$1 - \beta$	-3
low	search	Low	β	r_{search}
High	wait	High	1	r_{wait}
High	wait	Low	0	-
low	wait	High	0	-
low	Wait	Low	1	r_{wait}
low	recharge	High	1	0
low	recharge	Low	0	-



Markov decision process

- Bellman equation
 - It expresses a relationship between the value of a state and the values of its successor states.
 - The Bellman equation averages over all the possibilities, weighting each by its probability of occurring.
 - It states that the value of the start state must equal the (discounted) value of the expected next state, plus the reward expected along the way.

$$v_{\pi} \doteq \sum_a \pi(a|s) \sum_{s',r} p(s',r|s,a)[r + \gamma v_{\pi}(s')]$$



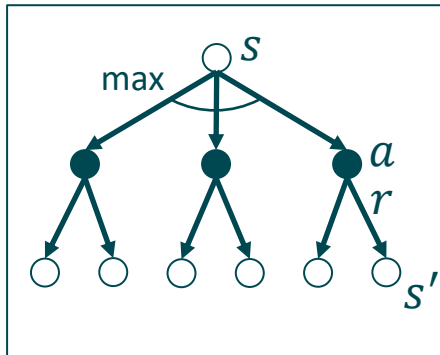
Backup diagram

Markov decision process

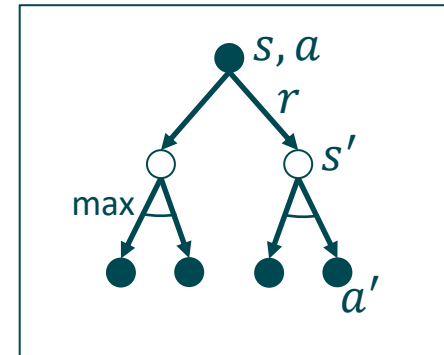
- Optimal policy and optimal action-value function

$$v_* \doteq \max v_\pi(s)$$

$$q_*(s, a) \doteq \max q_\pi(s, a) = \mathbb{E}[R_{t+1} + \gamma v_*(S_{t+1}) | S_t = s, A_t = a]$$



Backup diagram for v_*



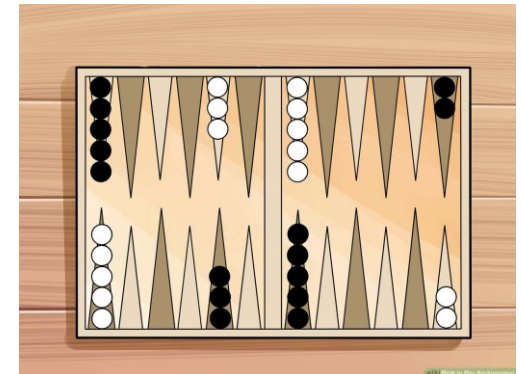
Backup diagram for q_*

Markov decision process

- Explicitly solving the Bellman optimality equation provides one route to finding an optimal policy, and thus to solving the reinforcement learning problem.
- But...
 - It may be considered analogous to an **exhaustive search**, whereby all **possible outcomes are evaluated** with regard to their probability of occurrence and their desirability in terms of the **expected rewards**.
 - Three assumptions that are rarely true in practice:
 1. We know exactly how the environment works.
 2. We have the enough computational resources to work out the solution.
 3. The Markov property.

Markov decision process

- Example Backgammon
 - First and third assumptions do not pose any issues in the context of backgammon.
 - The second is a major impediment. $\sim 10^{20}$ states!!
 - It is estimated that the process would take thousands of years on contemporary computers.
 - In Reinforcement Learning one typically has to settle for approximate solutions.



How to play Backgammon:

<https://www.youtube.com/watch?v=wEKlBAAsReg&t=89s>

Summary

- **Reinforcement Learning:**
 - RL is about learning how to behave to achieve a goal through interaction with an environment.
 - The agent and environment interact over discrete time steps.
 - The agent has full control and knowledge of its internal processes, but the environment is incompletely controllable and may not be fully known.
- **Key Components:**
 - **Policy:** A stochastic rule that determines the agent's actions based on states.
 - **Objective:** Maximize cumulative reward over time.
 - **Markov Decision Process (MDP):** A well-defined RL setup with transition probabilities. Finite MDPs have finite state, action, and reward sets.
- **Return and Task Types:**
 - **Return:** The function of future rewards the agent aims to maximize.
 - **Episodic Tasks:** Use undiscounted returns; interactions break into episodes.
 - **Continuing Tasks:** Use discounted returns; interactions continue indefinitely.
- **Value Functions and Optimality:**
 - **Value Functions:** Assign expected returns to states or state-action pairs based on a policy.
 - **Optimal Value Functions:** Represent the maximum expected return achievable by any policy.
 - **Optimal Policy:** A policy with optimal value functions. Multiple optimal policies can exist.
 - **Bellman Optimality Equations:** Conditions that optimal value functions must satisfy, used to derive optimal policies.
- **Knowledge and Computational Constraints:**
 - **Complete Knowledge:** The agent has a full and accurate model of the environment (e.g., MDP dynamics).
 - **Incomplete Knowledge:** The agent lacks a perfect environment model.
 - **Computational and Memory Constraints:** Limited computation and memory restrict the agent's ability to fully utilize models or store exact value functions, necessitating approximations.
- **Optimality:**
 - Optimality provides a theoretical framework for understanding RL algorithms.



**Mondragon
Unibertsitatea**

Escuela Politécnica
Superior

Eskerrik asko
Muchas gracias
Thank you

© 2026 Mondragon Unibertsitatea.

All rights reserved. This document and its contents are protected by intellectual property rights. Its total or partial reproduction, distribution, public communication, transformation, adaptation or creation of derivative works is expressly prohibited without the written consent of the owner.

Aitor Duo Zubiaurre

aduo@mondragon.edu

Olagorta Kalea, 26
48014 Bilbo (Bizkaia)

T. +34 670 254 555

www.mondragon.edu